

Практическая работа 7 по Информационным технологиям

Кадр стека

Как правило, подпрограмме необходимо хранить какие-то данные в памяти. Это, во-первых, сохраняемые значения регистров, во-вторых — параметры и возвращаемые значения подпрограммы, если их больше, чем соответствующих регистров, а в-третьих — разнообразные значения, для которых в процессе планирования вычислений не оказалось свободного регистра. Данные, которые подпрограмма временно хранит в памяти, называются локальными переменными.

Если подпрограмма использует много локальных переменных, описанная ранее конвенция оказывается довольно неудобной.

Локальные переменные характеризуются смещением относительно вершины стека. При добавлении в стек (например, очередной локальной переменной) все эти смещения меняются и это надо всегда помнить программисту (чем-то напоминает программирование в кодах). Допустим, мы положили в стек некоторое значение (например, 5). Теперь оно доступно по адресу (**sp**). После того, как мы положим в стек следующее значение (скажем, 100500), 5 становится доступно по адресу **4(sp)** (стек растёт вниз), а 100500 — (**sp**). Добавление третьего снова изменит смещения относительно **sp**. Каждый раз, помещая в стек или снимая оттуда значения, мы вынуждены использовать новые смещения для остальных данных в стеке.

Для того, чтобы перевести **sp** в состояние, пригодное для восстановления **ra**, необходимо вести (в уме?) учёт всех сохранённых на стеке регистров и расположенных там локальных переменных, и снимать со стека ровно столько значений, сколько успели туда положить.

Возникает идея хранить в специально отведённом регистре (он называется *Frame Pointer*, указатель кадра - **s0**) состояние **sp** на момент вызова функции. Тогда для восстановления **sp** перед возвратом из функции достаточно будет переложить туда значение **s0**. Предыдущее значение **s0** при этом тоже надо будет сохранять на стеке, т. к. кадр локализован для каждого вызова функции.

Как обычно, в конвенции присутствует *преамбула* и восстановление стека, то есть код, дополнительно выполняемый вызывающей стороной, а также *пролог* и *эпилог*, то есть код, дополнительно выполняемый подпрограммой. Они заметно расширились — за удобство платим объёмом кода.

1. (преамбула) Передаваемые подпрограмме значения надо заносить в регистры **a***
2. (преамбула) Если передаваемых параметров больше 8, все остальные необходимо положить в стек в обратном порядке (последний, предпоследний, ... десятый, девятый)
3. Вызов п/п должен производиться командой **jal** или **jalr**
4. Никакая вызываемая подпрограмма не модифицирует содержимое стека выше того указателя, который она получила в момент вызова (напомним, что «выше» — это ближе к началу)
5. (пролог) Подпрограмма обязана сохранить на стеке значения **ra** (адреса возврата) и **s0** (адрес предыдущего кадра)

6. (пролог) Значение **sp**, каким оно было на момент вызова подпрограммы, она обязана записать в **s0**
 7. (пролог) Подпрограмма обязана сохранить на стеке значения **ra** и всех *используемых ею* регистров типа **s***
 8. (пролог) Подпрограмма обязана выделить на стеке память для всех локальных переменных
 9. Подпрограмма *может* хранить на стеке произвольное количество временных данных. Количество этих данных и занимаемого ими места на стеке не оговаривается и *может меняться* в процессе работы подпрограммы. Такие данные, например, могут образоваться в преамбуле вызова подпрограммы при размещении параметров в стеке. Локальные переменные *рекомендуется* размещать в пределах кадра.
 10. Возвращаемое подпрограммой значение надо заносить в регистры **a0** - **a1**
 11. (эпилог) Подпрограмма обязана восстановить из стека значения всех сохраняемых регистров
 12. (эпилог) Подпрограмма обязана восстановить из стека значение **ra** и указателя кадра **s0**
 13. (эпилог) Подпрограмма обязана восстановить значение **sp**
 14. Возврат из подпрограммы должен производиться командой **ret**
 15. (очистка стека) Если передаваемых параметров было больше 8, необходимо поднять указатель стека на нужное смещение
- Согласно такой конвенции:
- смещение относительно **указателя кадра** всех используемых в функции локальных данных *постоянно*. Можно, например, задать им имена с помощью **.equ**. Если локальных переменных много и они часто, мнемоника сильно упрощает работу с исходным кодом;
 - восстановление стека используется перед выходом не зависит от актуального размера кадра, достаточно, чтобы не изменился;
 - сброс кадра частично защищает программу от сноса стека (в большую сторону): если сам **указатель кадра** не испорчен, и ошибочное изменение стека не добралось до сохранённых значений, ошибку можно локализовать при отладке. Такая ошибка возникает, например, если разместить какие-то данные в стеке, и забыть их оттуда снять;
 - дополнительные входные данные, передаваемые подпрограмме через стек, также доступны по *фиксированным смещениям* относительно **s0**, причём, в силу порядка, девятый параметр будет просто лежать по адресу **4(s0)**, десятый — **8(s0)** и т. д. (напоминаем, стек растёт вниз);
 - что не менее важно, при отладке программы с *неиспорченным* стеком, можно автоматически отследить т. н. цепочку вызовов (call trace) — последовательность вызовов подпрограмм, которая привела к вызову текущей. В самом деле, **указатель кадра** в этой конвенции указывает на место в стеке, где хранится *предыдущее значение s0*, а оно указывает на пред-предыдущее и т. д.
- цепочка вызовов — это обычный *связный список*, начало которого *сейчас* хранится в **s0**, и работать с ним можно соответственно.

Обратите внимание на то, что **fp** и **s0** — это один и тот же регистр. В некоторых конвенциях может быть не предусмотрено кадра — или договорённость о нём не будет использовать выделенный регистр (например, если кадр *строго фиксированного* размера).

Стоит заметить, что и эта конвенция

- не универсальна
- не для всего удобна.

Подпрограмма с использованием кадра

Это подпрограмма, сортирующая методом обмена массив целых размером **a0** байтов (т. е. **a0/4** элементов), лежащий по адресу **a1**. Подпрограмма не концевая — она вызывает ещё одну подпрограмму, поиск минимального элемента в массиве.

- Подпрограмма использует два сохраняемых регистра **s** для хранения счётчика и адреса обмена, поэтому эти два регистра сохраняются в кадре вместе с **fp** и **ra**.
- Подпрограмма запоминает адрес и размер массива в локальных переменных, чтобы затем вернуть их, согласно конвенции, в **a0** и **a1**. Конечно, можно было бы и в сохраняемые регистры спасти эти значения, но чего не сделаешь ради удобного примера.
- Все смещения заданы .equ-константами, в том числе и размер кадра **FSIZE**, который просто совпадает со смещением последней локальной переменной.

```
# Смещение в кадре для сохраняемых регистров относительно fp
.equ    S1      -12      # s*
.equ    S2      -16
.equ    ARR     -20      # Смещения в кадре
.equ    SIZE    -24      # для локальных переменных
.equ    FSIZE   28       # Размер кадра (+ 4 на сам fp)
# Смещение в кадре для ra и fp относительно sp
.equ    RA      24       # ra (к сожалению, Ripes не умеет в FSIZE-4)
.equ    FP      20       # fp
sort:   # сортировка массива
        # Вход: a0 — размер массива, a1 — начало массива
        # Выход: a0 — размер массива, a1 — начало массива (того же!)
        addi    sp sp -FSIZE    # Заводим кадр
        sw      ra .RA(sp)      # Сохраняем ra
        sw      s0 .FP(sp)      # Сохраняем fp
        addi    s0 sp FSIZE     # Значение sp при входе в sort
        sw      s1 S1(s0)       # Теперь у нас есть fp, сохраняем s1
        sw      s2 S2(s0)       # сохраняем s2
        sw      a0 SIZE(s0)     # Размер массива
        sw      a1 ARR(s0)      # Адрес массива
        mv      s2 a0           # Регистры типа s* не меняются
        mv      s1 a1           # при вызовах функций
fnext:  mv      a0 s2
        mv      a1 s1
        jal     getmin
        lw      t0 0(a0)
```

```

lw t1 0(s1)
sw t0 0(s1)
sw t1 0(a0)
addi s1 s1 4
addi s2 s2 -4
bgtz s2 fnext
lw a0 SIZE(s0)      # Обращаемся к локальным переменным
lw a1 ARR(s0)
lw s2 S2(s0)        # Восстанавливаем регистры
lw s1 S1(s0)
lw ra RA(sp)        # Восстанавливаем адрес возврата
lw s0 FP(sp)        # Восстанавливаем кадр
addi sp sp FSIZE    # Восстанавливаем стек
ret

```

Концевая подпрограмма не использует кадр, так что формально она не соответствует конвенции. Несмотря на то, что основные договорённости не нарушаются (все сохраняемые регистры целы, стек не тронут), отладка такой подпрограммы может представлять известную трудность по сравнению с отладкой подпрограммы с кадром, если всё-таки в ней появляются локальные переменные, сохранение регистров и т. п. Впрочем, наша концевая подпрограмма и стек не использует:

```

getmin: # концевая подпрограмма
        # a0 — размер массива
        # a1 — начало массива
        # a0 — адрес минимального элемента
        lw      t0 0(a1)
        mv      t2 a1
gmnext: addi    a0 a0 -4
        addi    a1 a1 4
        blez    a0 gmfin
        lw      t1 0(a1)
        bge     t1 t0 gmnext
        mv      t0 t1
        mv      t2 a1
        j       gmnext
gmfin:  mv      a0 t2
        ret

```

```

# Вызывающая программа с выводом
.data
.equ    SZ 20
Arr:    .word  SZ
EndArr: .align 2
.text

```

```

main:   la      t1 Arr
        la      t2 EndArr
input:  li      a0 5
        sw      a0 0(t1)
        addi    t1 t1 4
        bgtu    t2 t1 input
        li      a0 SZ
        la      a1 Arr
        jal     sort
        la      t1 Arr
        la      t2 EndArr
output: li      a7 1
        lw      a0 0(t1)
        ecall
        li      a0 '\n'
        li      a7 11
        ecall
        addi    t1 t1 4
        bgtu    t2 t1 output

        li      a7 10
        ecall

```

Обратите внимание на строгую реализацию принципа «*сначала* переставим стек, *потом* будем сохранять значения».

Задание 1

Напишите программу для сортировка обменом. Вначале программа читает как константу из памяти натуральное число N — количество строк, а потом — N строк. Затем надо вывести эти строки сначала в порядке, указанном в программе, а потом — в лексикографическом порядке возрастания. Длина строк не должна превышать 80 символов и не должна быть меньше 32 символов.

Число N зависит от младшего разряда номера студ. билета m :

если $m < 4$, то $N = m + 7$;

если $m > 7$, то $N = m + 3$.

Например, для студ. билета 431630 m будет равно 0, а $N = 7$:

```

.equ      N      7
str1:     .string yufngukfnxdttthfjxtykxutoucuykvhetnymmkk1
str2:     .string hjkhjkhjkhjkhghjghgjghkgghjghjghghjghjgh

```

```
str3:    .string  akjhgdsrtyhwaseasefaefasefasrgdfthfh
str4:    .string  ercyjcfjcfjcfjgjdxdzeerzssezfszerzsefzsefz
str5:    .string  uyfftjfyfykvjftffhfnvgvthvjyjnvyjvjvjjv
str6:    .string  ityuipcfjyjcfjtidrtffjgcykigykcgyjgykgykgy
str7:    .string  ohmcgncfhfnoiuhobobobhuobhulbhuobholbh
```

Программа должна напечатать:

```
yufngukfnxdtthfjxtykxutoucuylvhetnymmkkl
hjkjhkhjkhjkhghjghgjghkgghjghjghghjghjgh
akjhgdsrtyhwaseasefaefasefasrgdfthfh
ercyjcfjcfjcfjgjdxdzeerzssezfszerzsefzsefz
uyfftjfyfykvjftffhfnvgvthvjyjnvyjvjvjjv
ityuipcfjyjcfjtidrtffjgcykigykcgyjgykgykgy
ohmcgncfhfnoiuhobobobhuobhulbhuobholbh
N = 7
akjhgdsrtyhwaseasefaefasefasrgdfthfh
ercyjcfjcfjcfjgjdxdzeerzssezfszerzsefzsefz
ityuipcfjyjcfjtidrtffjgcykigykcgyjgykgykgy
hjkjhkhjkhjkhghjghgjghkgghjghjghghjghjgh
ohmcgncfhfnoiuhobobobhuobhulbhuobholbh
uyfftjfyfykvjftffhfnvgvthvjyjnvyjvjvjjv
yufngukfnxdtthfjxtykxutoucuylvhetnymmkkl
```

Используйте структуру данных: таблица строк. Также понадобится подпрограмма `strget`. Сначала создаём массив на N адресов, сохраняем туда N результатов вызова п/п `strget`, а затем сортируем выбором при помощи п/п `strcmp`.

Контрольные вопросы:

1. Какой регистр обычно отвечает за хранение кадра стека?
2. Что должна сделать подпрограмма при возвращении со значением кадра стека?
3. Где надо хранить локальные переменные?